UNIT-II

Dr. U.Padmavathi

Department of Computer Science & Engineering
School of Engineering

Shiv Nadar University Chennai

SHIV NADAR
UNIVERSITY
CHENNAI

Process

- What to call all CPU activities?

- Batch system – jobs
- Time-shared system – tasks

- All these activities are called as processes.

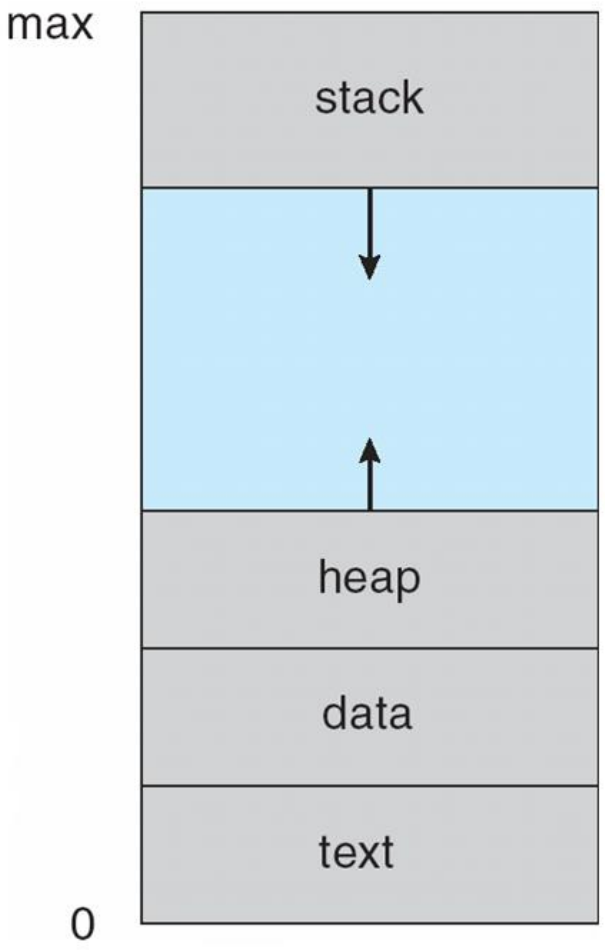
The Process

- A process is a program in execution.

- A process includes text section, data section, heap and stack.

- Text section – program code
- Data section – global variables
- Heap – dynamically allocated memory
- Stack – contains temporary data such as function parameters, return address and local variables.

Process in Memory



- A program is not a process.
- A program is a passive entity.
- A process is an active entity.
- A program becomes a process when an executable file is loaded into memory.
- By double-clicking icon or entering name of file.

- A process itself can be an execution environment for other code.

- Example: java programming environment
- An executable java program is executed within the java virtual machine.

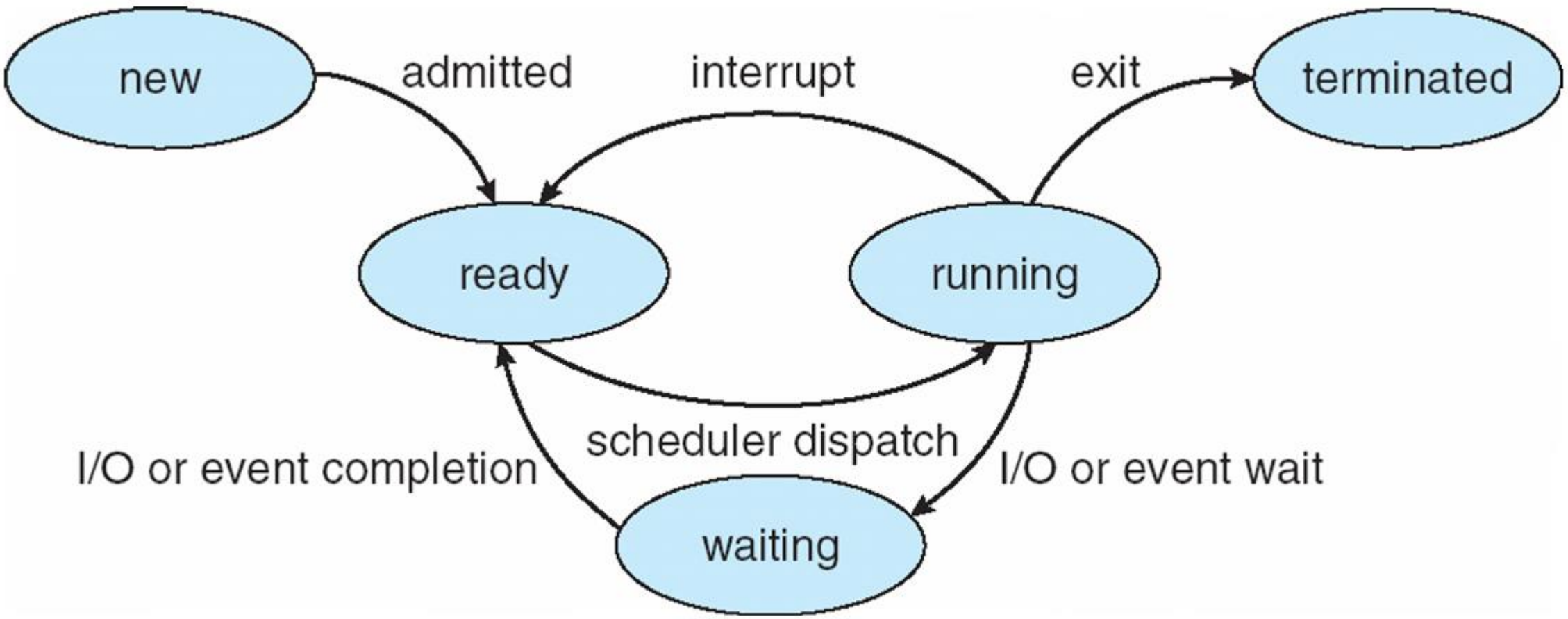
Process State

- As a process executes, it changes state.

- State – current activity of the process.

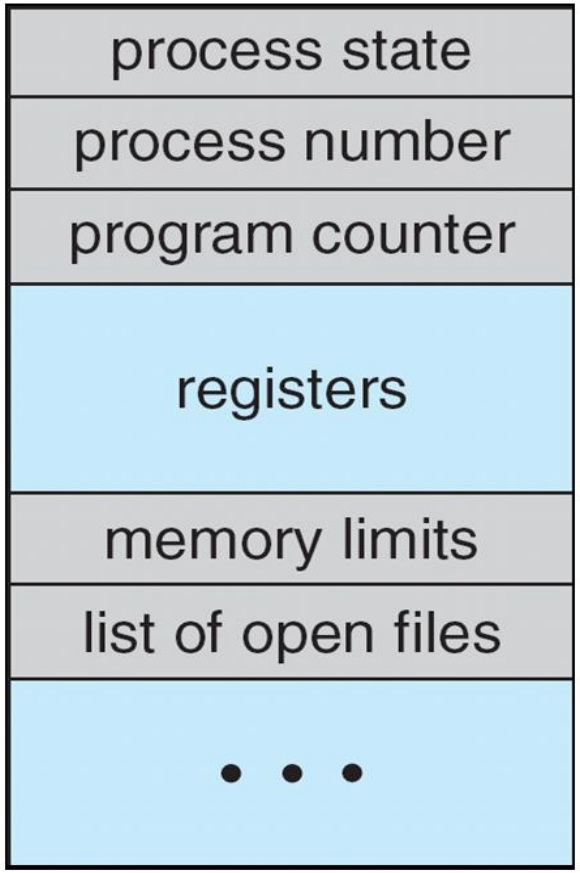
- New – created.
- Running – executed.
- Waiting – waiting for some event.
- Ready – waiting to be assigned to a processor.
- Terminated – finished execution.

Process State



Process Control Block

- Each process is represented in the OS by PCB.



PCB

- Process State
- Program Counter
- CPU registers – accumulators, index registers, stack pointers, general-purpose registers.
- CPU-Scheduling information – process priority, pointes to scheduling queues and scheduling parameters.
- Memory-mgt infn – value of the base and limit registers, page tables, segment tables.
- Accounting infn – amount of CPU used, time limits, account numbers, process numbers
- I/O Status – list of I/O devices allocated to a process,open files.

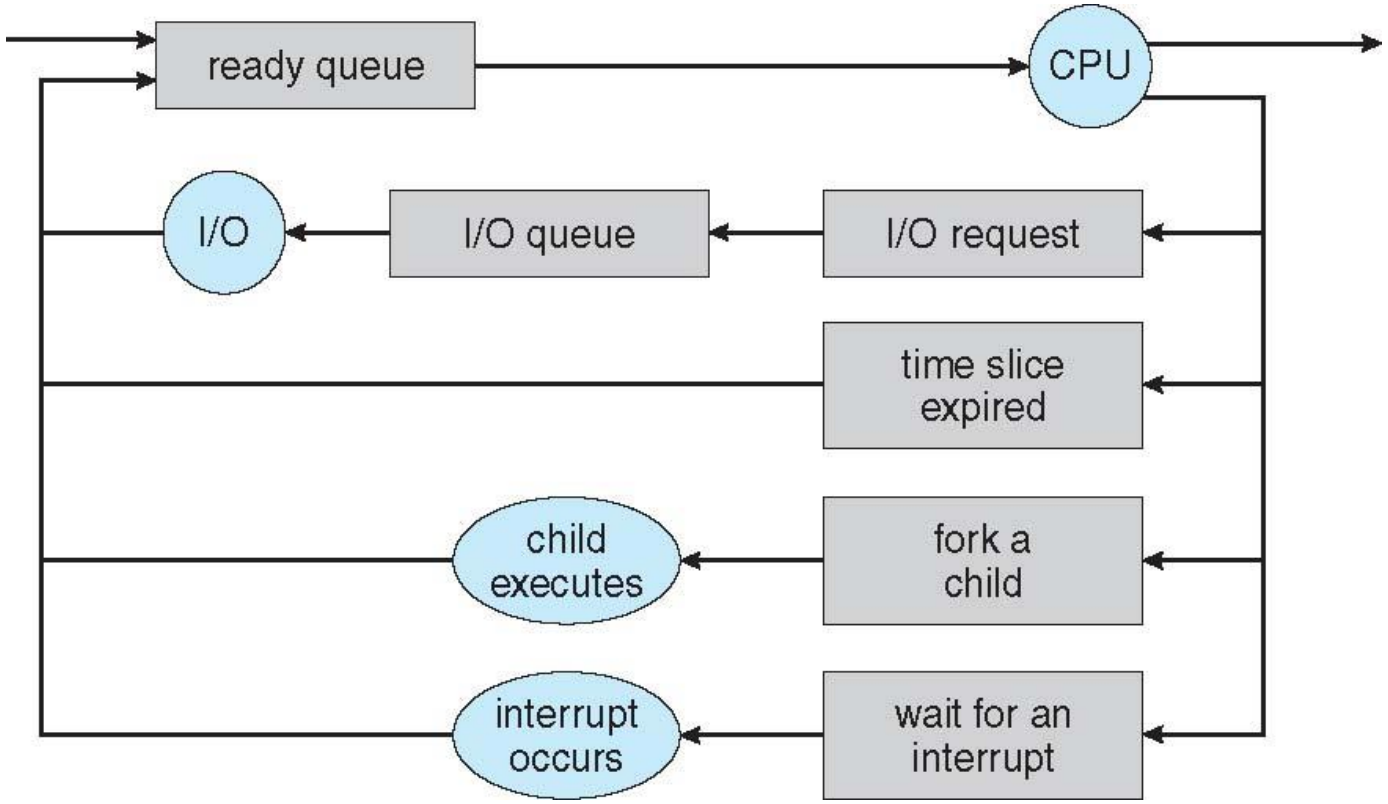
Threads

- A process is a program that performs a single thread of execution.
- Example: Word processor – type characters and run spell check.
- Modern operating systems – multiple threads of execution – more task at one time.

Process Scheduling

- Process scheduler – selects an available process for program execution on the CPU.
- When the process enter the system, they are put into a job queue.
- The process that are ready and waiting to execute are kept in the ready queue.
- List of processes waiting for a particular I/O device is put on device queue.

Queuing Diagram



Schedulers

- A process migrates among the various scheduling queues throughout its lifetime.

- Scheduler – responsible for selecting the process for scheduling.
- Long-term scheduler
- Short-term scheduler
- Medium-term scheduler

- Batch system – processes are spooled to mass storage device.
- Long-term scheduler – selects processes from this pool and load into memory.
- Short-term scheduler – selects from among the processes that are ready to execute and allocates the CPU to one of them.

- Short-term scheduler – executes frequently.
- It must be fast.

- Long-term scheduler – executes much less frequently.
- Is invoked only when the process leaves the system.
- Can take long time to decide which process should be selected for execution.
- Careful selection.

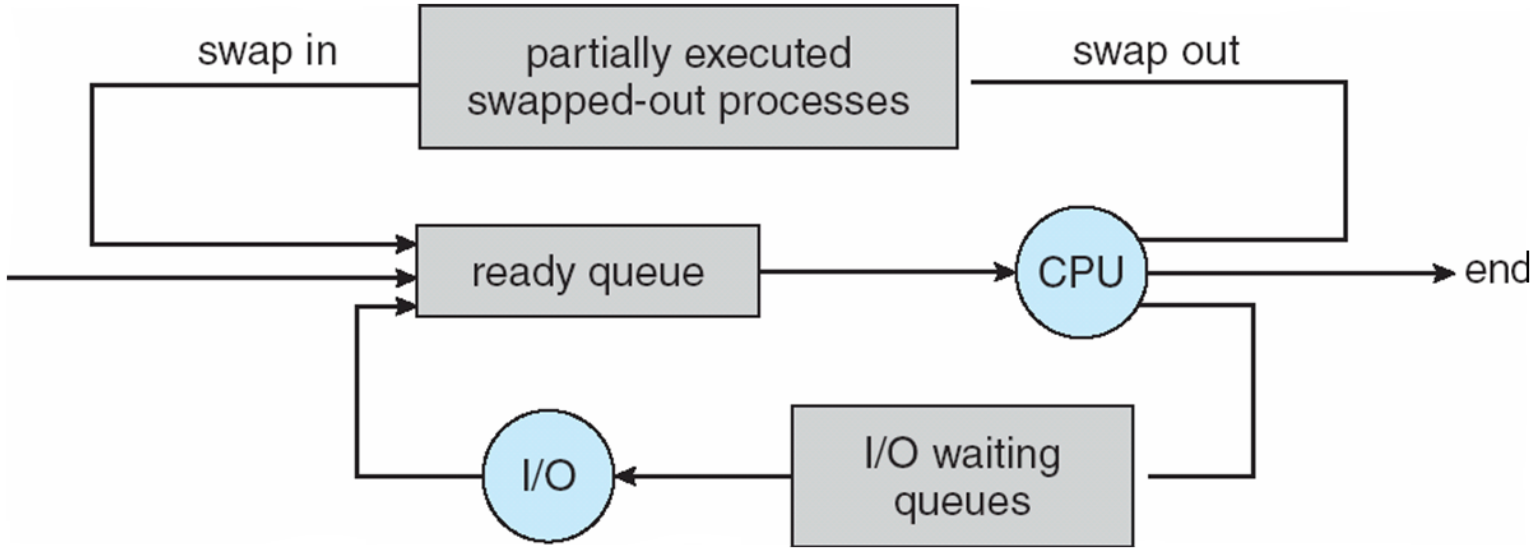
- Process – I/O bound or CPU bound.

- I/O bound – spend more time on I/O
- CPU bound – spends more time on computations.

- LTS – select a good process mix of I/O bound and CPU-bound – good system performance.

- Some operating systems have additional intermediate level of scheduling – medium-term scheduler.

- Medium-term scheduler – swapping – swap-in and swap-out process – to free up memory – improve the process mix.



Context Switch

- When an interrupt occurs, the system needs to save the context of the process running on the CPU.
- The context is represented in the PCB of the process.
- It includes the value of CPU registers, the process state, and memory-management information.
- Switching the CPU to another process requires performing a state save of the current process and a state restore of the different process. This is known as context switch.

- Context switch – saves the context of old process - Loads the context of the new process scheduled to run.

- Context-switch time is pure overhead.

- Context-switch time are highly dependent on hardware support.

Operations on Processes

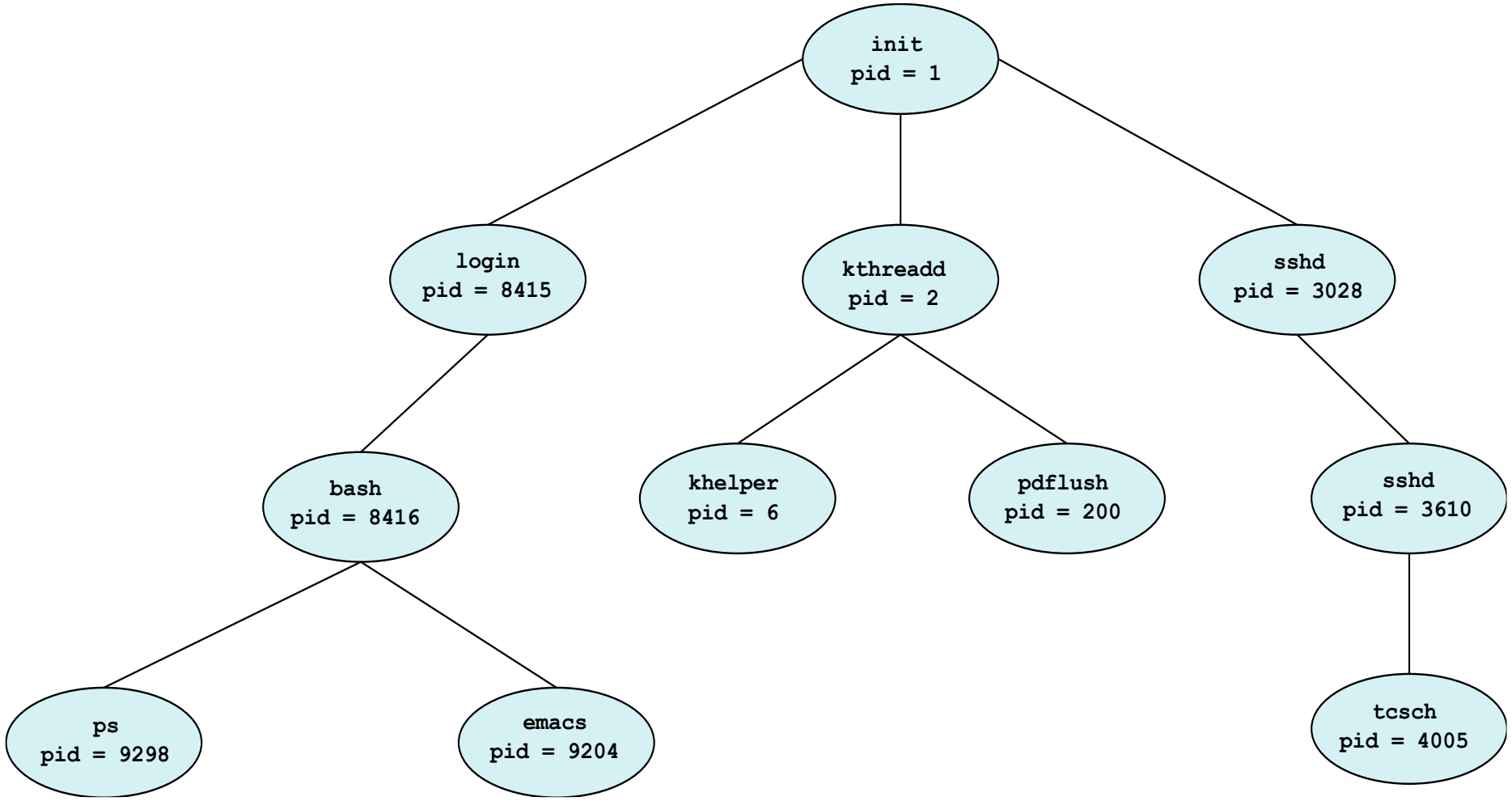
- Process Creation

- A process may create several new processes.
- The creating process – parent
- Created process – child

- Each of these new processes may create other processes forming a tree of processes.

- The processes are identified according to a unique process identifier (pid).
- Pid is an integer number.
- It provides a unique value and is used as an index to access various attributes of a process.

Tree of Processes on a typical Linux System



SHIV NADAR UNIVERSITY CHENNAI

- On UNIX and Linux systems, the list of processes can be obtained by ps command.

- Example: ps –e1
- List complete information for all processes currently active in the system.

- We can construct the Process tree by recursively tracing parent processes all the way to the init process.

- A child process obtain its resources directly from the operating system or it may be constrained to a subset of the resources of the parent process.

- The parent process along with supplying resources, may pass initialization data.

- Example: display the content of a file named image.jpg

- When a process creates a new process, two possibilities for execution exist:

- The parent continue to execute concurrently with its children.
- The parent waits until some or all of its children have terminated.

- Two-address space possibilities for the new process:

- The child process is a duplicate of the parent process
- The child process has a new program loaded into it.

Process Termination

- A process terminates when it finishes executing its final statement and asks the operating system to delete it using the exit () system call.

- All the resources of the process – including physical and virtual memory, open files and I/O buffers are deallocated by the operating system.

- A process can cause the termination of another process via an appropriate system call.

- A parent may terminate the execution of one of its children for a variety of reasons:

- The child has exceeded its usage of some of the resources that it has been allocated.
- The task assigned to the child is no longer required.
- The parent is exiting and the OS does not allow a child to continue if its parent terminates.

- Cascading termination – if a process terminates, then all its children must also be terminated. It is initiated by the operating system.

- In linux, a process is terminated using the exit () system call.

- A parent process wait for the termination of the child process using the wait () system call.

- When a process terminates, its resources are deallocated by the operating system.

- A process that has terminated, but whose parent has not yet called the wait (), is known as zombie process.

- Once the parent calls wait (), the process identifier of the zombie process and its entry in the process table are released.

- A parent process terminated without invoking wait (), leaves its child processes as orphans.

InterProcess Communication

- Processes executing concurrently in the OS may be either independent processes or cooperating processes.

- Independent – if it cannot affect or be affected by other processes.
- Do not share data with other processes.

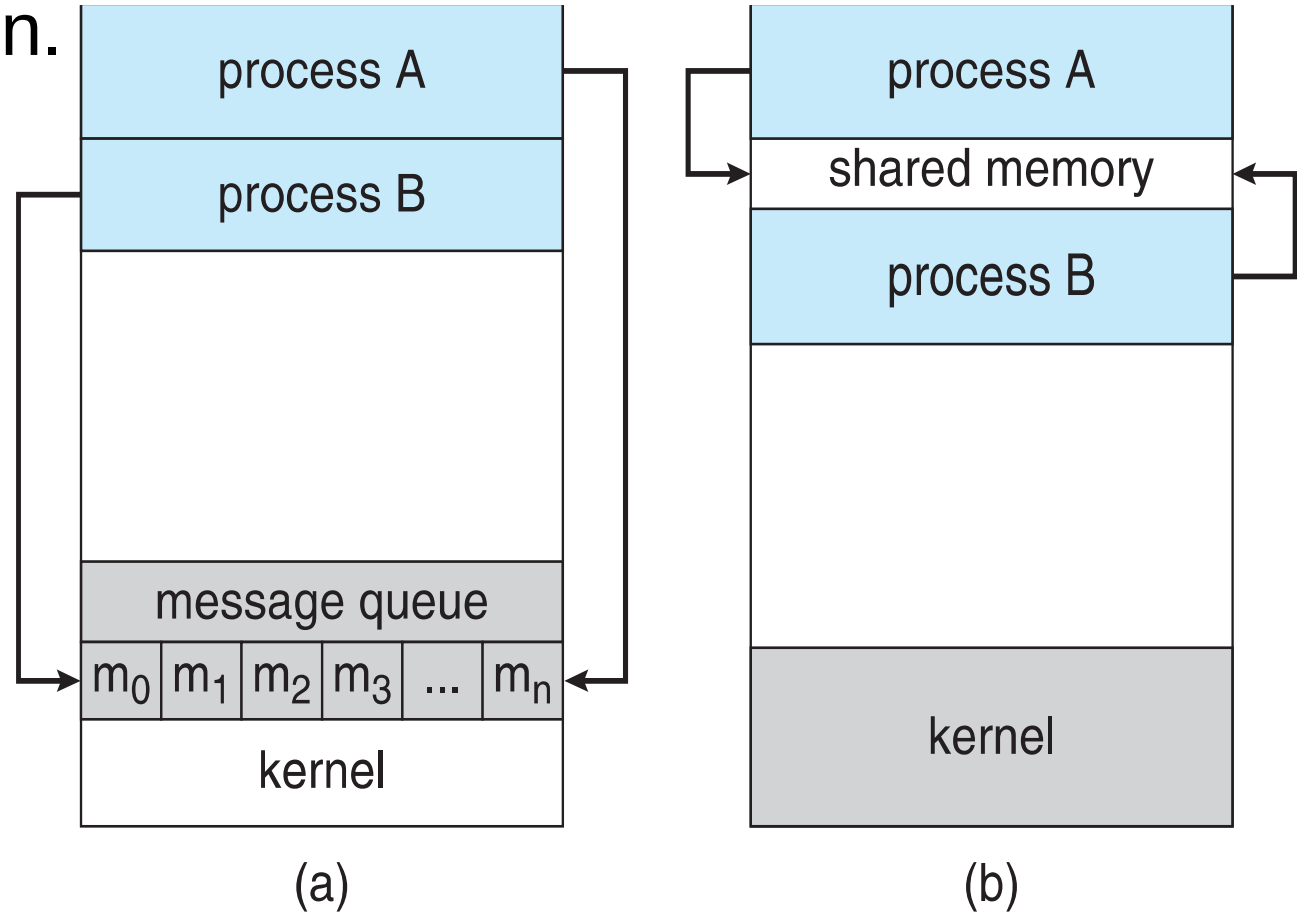
- Cooperating – if it can affect or can be affected by other processes.
- Shares data with other processes.

Reasons for providing an environment of process cooperation

- Information sharing
- Computation Speed up
- Modularity
- Convenience

- Cooperating process requires an IPC mechanism that will allow them to exchange data and information.

- Two ways of IPC:
- Shared memory - faster
- Message Passing



SHIV NADAR UNIVERSITY CHENNAI

Shared-Memory

- IPC using shared memory requires communicating processes to establish a region of shared memory.
- Shared-memory region resides in the address space of the process creating the shared-memory segment.
- Two or more processes can exchange information by reading and writing data in the shared areas.
- The form of the data and location are determined by these processes and are not under the operating system's control.

Producer Consumer Problem

- A producer process produces information that is consumed by a consumer process.
- Metaphor – client-server paradigm.
- Producer Consumer problem uses shared memory.
- To allow producer and consumer to run concurrently – a buffer of items –filled by producer and emptied by consumer.

- The producer and the consumer must be synchronized, so that the consumer does not try to consume an item that has not yet been consumed.

- Two types of buffer:
- Unbounded buffer – no limit on the size of the buffer
- Bounded buffer – fixed buffer size

Message Passing

- It allows the processes to communicate and synchronize their actions without sharing the same address space.
- It is useful in a distributed environment.
- Example: Internet chat

Operations

- Message passing facility provides at least two operations:
- Send (message)
- Receive(message)

- Message – fixed or variable – straightforward – complex – system level implementation.

- P and Q communicate – communication link – not the physical link – logical implementation.

Methods for logically implementing a link and send () /receive () operations

- Direct or indirect communication
- Synchronous or Asynchronous communication
- Auutomatic or explicit buffering

Naming

- Processes that communicate must have a way to refer to each other.
- Direct or Indirect

- Direct – each process must explicitly name the recipient or sender of the communication.
- Send(P, message)
- Receive (Q, message)

- A communication link in this scheme has the following properties:
- A link is established automatically – identity must be known.
- A link is associated with exactly two processes
- There exists exactly one link

- It exhibits symmetry in addressing – both sender and receiver must name the other to communicate.

- Asymmetry – sender names the recipient – recipient is not required to name the sender.

- Send(P, message)
- Receive(id, message)

- Disadvantage – limited modularity of the resulting process definition.

- Indirect Communication – messages are sent and received using mailboxes or ports.

- Each mailbox has a unique identification.

- Two processes can communicate only if they have shared mailbox.
- Send(A, message)
- Receive(A, message)

- In this, the communication link has the following properties:

- A link is established between a pair of processes only if both members of the pair have a shared mailbox.
- A link may be associated with more than two processes.
- Between each pair of communicating processes, a number of different links may exist, with each link corresponding to one mailbox.

- P1,P2,P3 – share mailbox A.
- P1 send.
- P2? Or P3? Which receive?

- Answer – depends on which of the following methods we choose.
- Allow a link to be associated with two processes at most.
- Allow at most one process at a time to execute receive ().
- Allow the system to select arbitrarily.

Synchronization

- Communication between processes takes place through calls to send () and receive () primitives.

- Message passing – blocking or non-blocking – synchronous or asynchronous.

- Blocking send – sending process is blocked until the message is received by the receiving process or by the mailbox.
- Nonblocking send – sending process sends the message and resumes operation.
- Blocking receive – the receiver blocks until a message is available.
- Nonblocking receive – the receiver retrieves either a valid message or a null.

Buffering

- Direct or indirect – messages exchanged by communicating processes reside in a temporary queue.

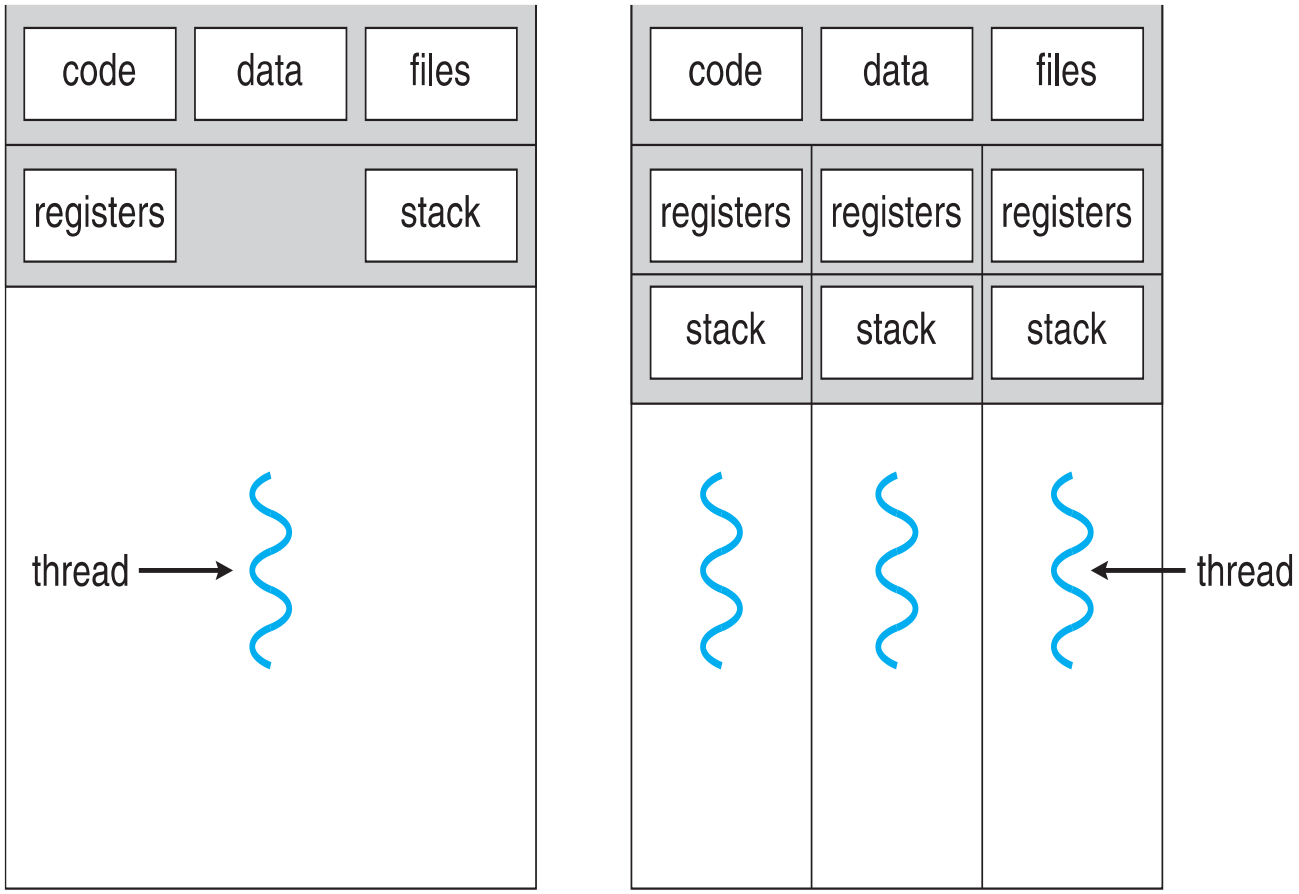
- Queues implemented in three ways:
- Zero capacity

- Bounded capacity

- Unbounded capacity

Thread

- A thread is a basic unit of CPU utilization.
- Thread ID, program counter, register set and stack.
- It shares with other threads belonging to the same process its code section, data section, and other operating-system resources, such as open files and signals.



single-threaded process

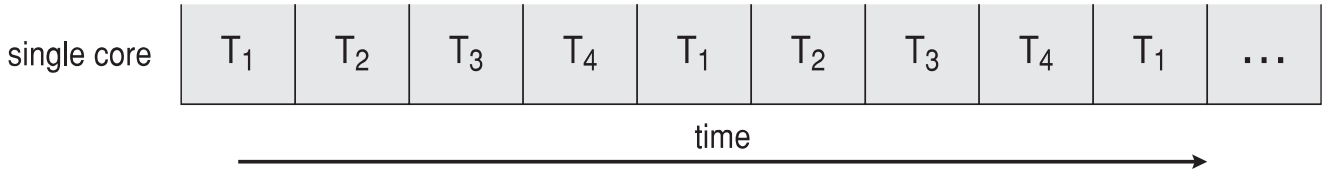
multithreaded process

Multicore Programming

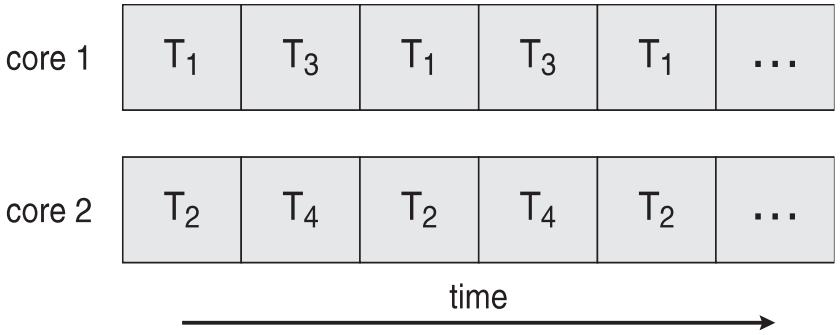
- Place multiple computing cores on a single chip. Each core appears as a separate processor to the operating system – Multicore or Multiprocessor systems.

- Multithreaded programming provides a mechanism for more efficient use of these multiple computing cores and improved concurrency.

- Concurrent execution on a single core system



- Parallel execution on a multicore system



- Before the advent of multicore architectures, most computer systems had single processor.

- CPU schedulers provide the illusion of parallelism.

- Thread performance is improved by adding hardware.

- Intel CPU – 2 threads
- Oracle T4 CPU – 8 threads

Programming Challenges

- Multicore system – Place pressure on system designers and application programmers – to make better use of multiple computing cores.

- Designer of OS – write scheduling algorithms – allow parallel execution
- Appln Programmer – modify existing programs and design new programs that are multithreaded.

- Five areas present challenges in programming for multicore systems:

- Identifying tasks
- Balance
- Data Splitting
- Data dependency
- Testing and Debugging

- Software developers – argue – advent of multicore systems – entirely new approach to designing software systems.

Types of Parallelism

- Data Parallelism
- Task Parallelism

- Data Parallelism – distributing subsets of the same data across multiple computing cores and performing the same operation on each core.
- Example: summing the contents of an array of size N.

- Task Parallelism – distributing tasks across multiple computing cores.
- Each thread is performing a unique operation.
- Different threads – operate – same data or different data.

Multithreading Models

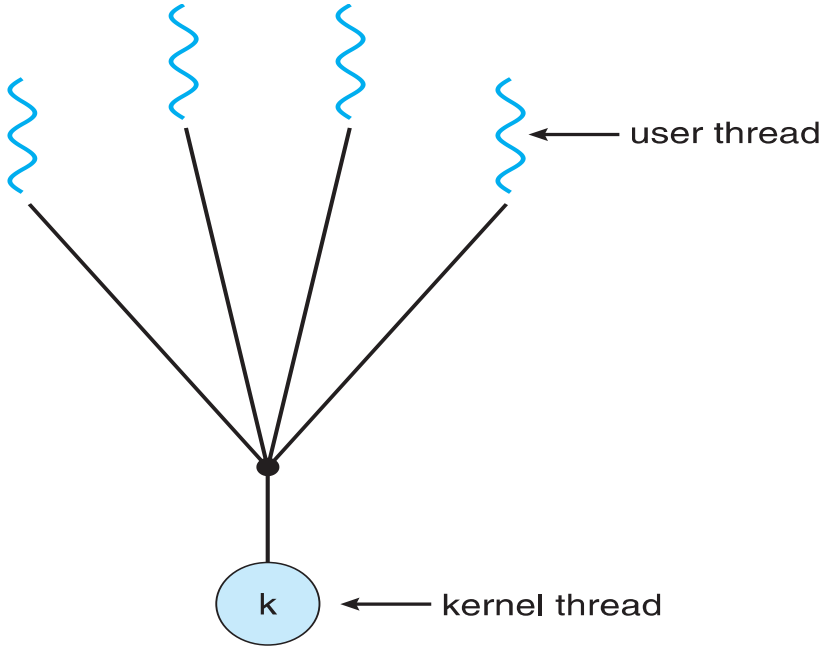
- User threads – user level – above kernel – without kernel support
- Kernel threads – Kernel level – managed by OS

- A relationship must exist between user thread and kernel thread.
- Many-to-one
- One-to-one
- Many-to-many

Many-to-one

- Maps many user-level thread to one kernel-level thread.

- Thread management is done by the thread library in the user space.



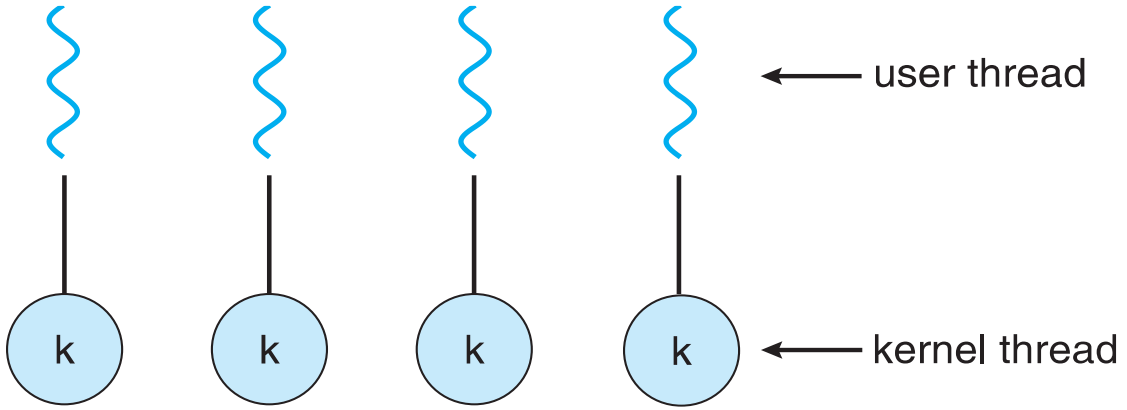
- The entire process will block if a thread makes a blocking system call.

- Only one thread can access the kernel at a time – Multiple threads cannot run in parallel on multicore systems.

- Example: Green threads – library for solaris systems.
- Inability to take advantage of multiple processing cores.

One-to-one model

- Maps each user thread to a kernel thread.
- Provides more concurrency by allowing another thread to run when a thread makes a blocking system call.
- Allows multiple threads to run in parallel.
- Creating a user thread requires creating the corresponding kernel thread – overhead – restrict the number of threads.



SHIV NADAR UNIVERSITY CHENNAI

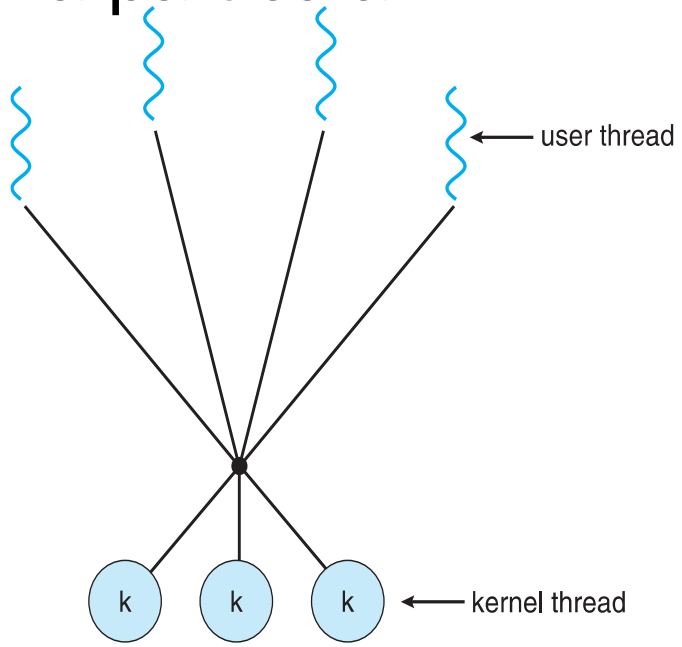
Many-to-Many model

- Multiplexes many user-level threads to a smaller or equal number of kernel threads.

- Number of kernel threads may be specific to either a particular application or a particular machine.

- Developers can create as many user threads

as necessary.



SHIV NADAR
UNIVERSITY
CHENNAI

Threading Issues

- Issues to be considered while designing multithreaded program

- Fork () and exec () system calls
- Signal Handling
- Thread Cancellation
- Thread-Local Storage
- Scheduler Activations

Fork () and exec () system calls

- Semantics of fork () and exec () system calls change in a multithreaded program.

- fork()
- does the new process duplicate all the threads ?
- or the new process is single threaded?

- Depends on the application.

Signal Handling

- A signal is used in UNIX systems to notify a process that a particular event has occurred.

- Signal may be synchronous or asynchronous.

- All signals follow the same pattern:
- A signal is generated by the occurrence of a particular event.
- The signal is delivered to a process.
- Once delivered, the signal must be handled.

- Synchronous signal – signals delivered to the same process that performed the operation that caused the signal.
- Example: Illegal Memory access, division by zero

- Asynchronous signal – signal is generated by an event external to a running process, that process receives the signal asynchronously.
- Example: terminating a process with specific keystroke, having a timer expire.

- A signal may be handled by one of two possible handlers:
- A default signal handler.
- A user-defined signal handler.

- Handling signals in single-threaded program is straightforward.
- In multi-threaded program, it is complex – where to deliver the signal.

- Following options exist:

- Deliver the signal to the thread to which the signal applies.
- Deliver the signal to every thread in the process.
- Deliver the signal to certain threads in the process.
- Assign a specific thread to receive all signals for the process.

Thread Cancellation

- Terminating a thread before it has completed.
- Example: multiple threads searching a database.
- User presses a button that stops loading a web page.

- A thread that is to be cancelled is referred as target thread.

- Cancellation of a target thread occurs in two scenarios:
- Asynchronous cancellation
- Deferred cancellation

- Difficulty occurs in asynchronous cancellation:
- Resources are allocated
- Cancelling a thread in the midst of updating

- With deferred cancellation, the target thread checks the flag to determine whether it can be cancelled.

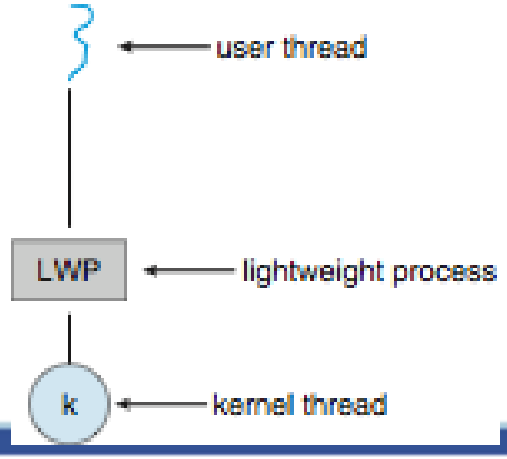
- Once the thread receives the cancellation request, all its resources will be released and the thread is terminated.

Thread-Local Storage

- Threads belonging to a process share the data of the process.
- In some situations, each thread needs its own copy of the data – Thread Local Storage.
- Example: Transaction Processing System.
- Issue is confusing TLS with Local variables.

Scheduler Activations

- Final issue – communication between the kernel and the thread library.
- Kernel threads must be dynamically adjusted to ensure the best performance.
- Many systems place an intermediate data structure between the user and kernel threads.



- Data structure – Lightweight process – virtual processor on which the user thread run.

- If kernel thread blocks, LWP blocks as well, user-level thread also blocks.

- An application that is I/O intensive may require multiple LWPs to execute.

- Example: five different file-read requests – five LWPs are needed

Process Scheduling

- Single processor – one process run
- Objective of multiprogramming – have some process running at all times – maximize CPU utilization.
- Scheduling – function of operating system.

CPU-I/O Burst cycle

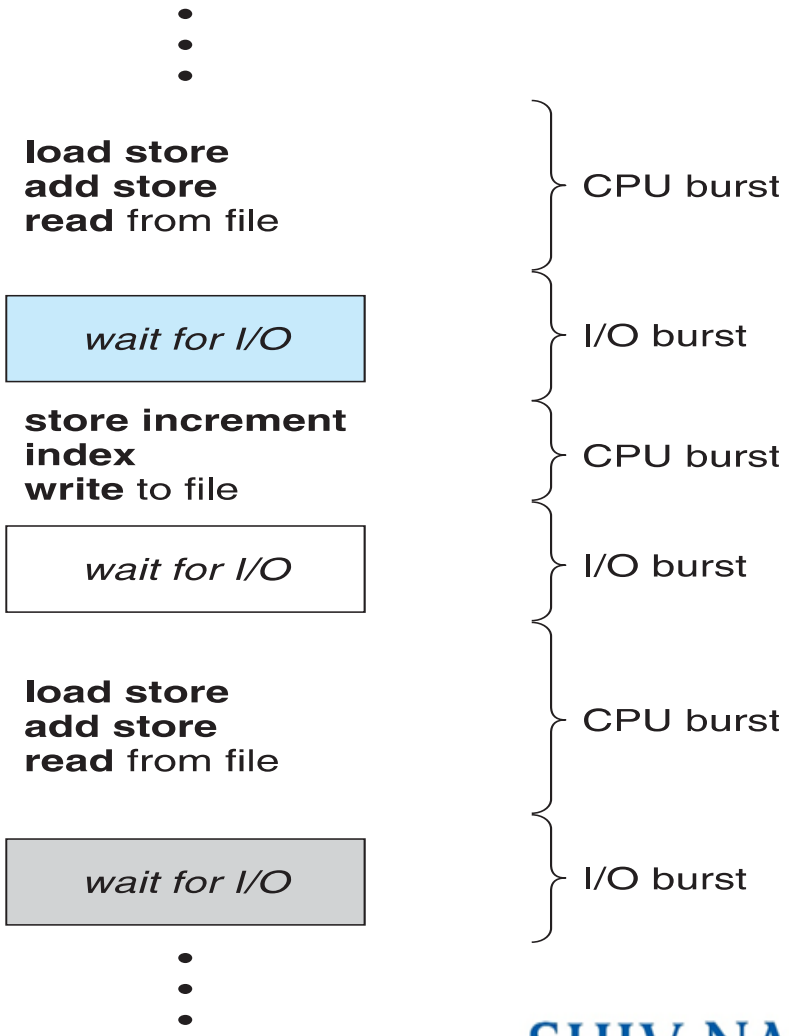
- Process execution -

cycle of CPU execution and I/O wait.

- CPU burst followed by I/O burst

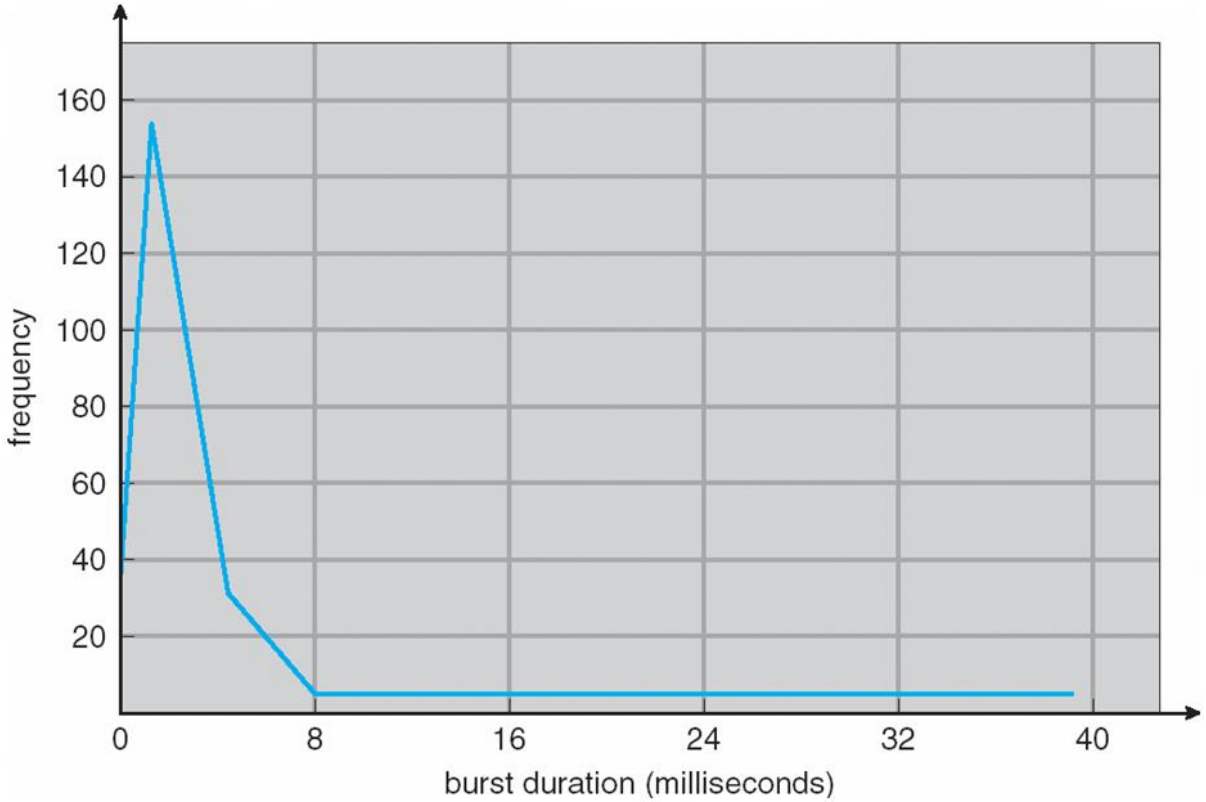
- Final CPU burst ends with system

request to terminate execution.



- Durations of CPU bursts have been measured extensively.

- It tends to have a frequency curve similar to the one shown below.



- Large – Short CPU bursts
- Small – Long CPU bursts

- It is important in the selection of an appropriate CPU-scheduling algorithm.

CPU Scheduler

- Short-term scheduler

Preemptive scheduling

- CPU-scheduling decisions may take place under the following four circumstances:

- When a process switches from the running state to the waiting state.
- When a process switches from the running state to the ready state.
- When a process switches from the waiting state to the ready state.
- When a process terminates.

- When scheduling takes place under conditions 1 and 4, then it is nonpreemptive otherwise it is preemptive.

- Nonpreemptive – once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU either by terminating or by switching to the waiting state.
- Example: Microsoft Windows 3.x.

- Preemptive scheduling – results in race condition – inconsistent state.

Dispatcher

- One of the component in CPU-scheduling function.

- It is the module that gives control of the CPU to the processes selected by the short-term scheduler.

- It involves the following functions:
- Switching context
- Switching to user mode
- Jumping to the proper location in the user program to restart that program.

- Dispatcher should be as fast as possible.

- The time it takes for the dispatcher to stop one process and start another running is known as the dispatch latency.

Scheduling criteria

- Many criteria have been suggested for comparing CPU-Scheduling algorithms.

- CPU utilization
- Throughput
- Turnaround time
- Waiting time
- Response time

Scheduling algorithms

- It deals with the problem of deciding which of the processes in the ready-queue is to be allotted the CPU.

- First-come, First-Served
- Shortest-Job First
- Priority Scheduling
- Round-Robin
- Multilevel queue
- Multilevel Feedback Queue

First-Come, First-Served Scheduling

- Simplest scheduling algorithm.
- The process that requests the CPU first is allocated the CPU first.
- The average waiting time is quite long.
- FCFS algorithm in non preemptive.
- There is a convoy effect – all the other processes wait for the one big process to get off the CPU.

Shortest-Job First Scheduling

- It associates with each process the length of the CPU burst.
- When the CPU is available, it is assigned to the process that has the smallest CPU burst.
- If the CPU bursts of two processes are same, then FCFS is used.
- It gives the minimum average waiting time.
- SJF can be preemptive or non preemptive

Priority Scheduling

- A priority is associated with each process, and the CPU is allocated to the process with the higher priority.
- Equal priority processes are scheduled in FCFS order.
- SJF algorithm is a priority algorithms where the priority is inverse of the CPU burst.
- Priority scheduling can be either preemptive or non preemptive.

- A majority problem with priority scheduling is indefinite blocking or starvation.

- Leaving the low-priority processes to wait indefinitely.

- Solution – aging – gradually increasing the priority of the processes that wait in the system for a long time.

Round-Robin Scheduling

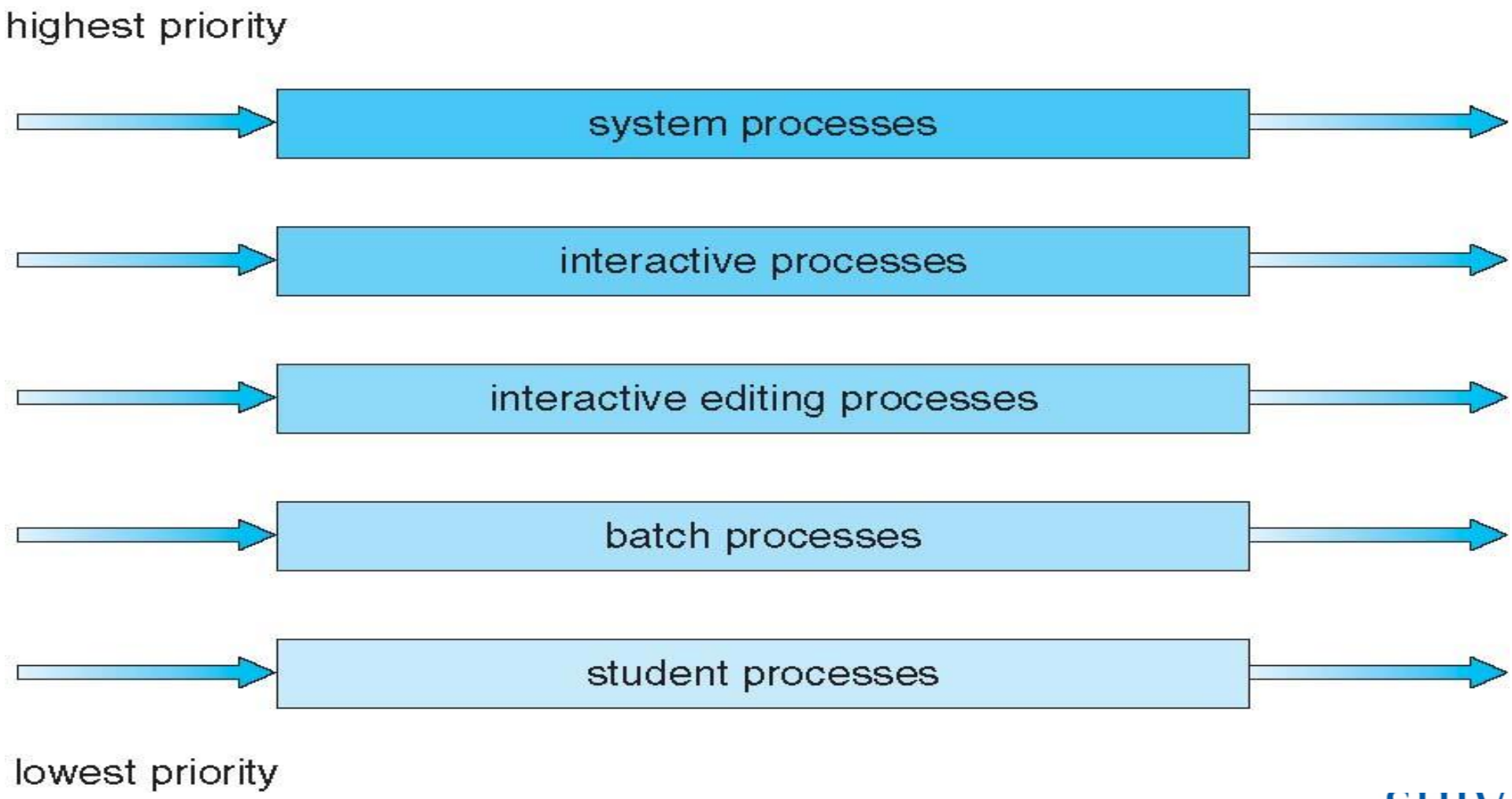
- Designed especially for time-sharing systems.
- Similar to FCFS, but pre-emption is added to enable the system to switch between processes.
- A small unit of time, called a time quantum or time slice is defined.
- The CPU scheduler goes around the ready queue, allocating the CPU to each process for a time interval of up to 1 time quantum.

- If there are n processes in the ready queue and the time quantum is q, then each process must wait no longer than (n-1) * q time units until its next time quantum.

- The performance of the RR algorithm depends heavily on the size of the time quantum.

Multilevel Queue Scheduling

- It partitions the ready queue into several separate queues.
- Processes are classified into different groups: foreground, background.
- The processes are permanently assigned to one queue, generally based on some property of the process, such as memory size, process priority or process type.
- There must be scheduling among the queues, which is commonly implemented as fixed-priority preemptive scheduling.


highest priority
system processes
interactive processes
interactive editing processes
batch processes
student processes
lowest priority

Multilevel Feedback queue Scheduling

- Multilevel queue scheduling – low overhead of scheduling but inflexible.

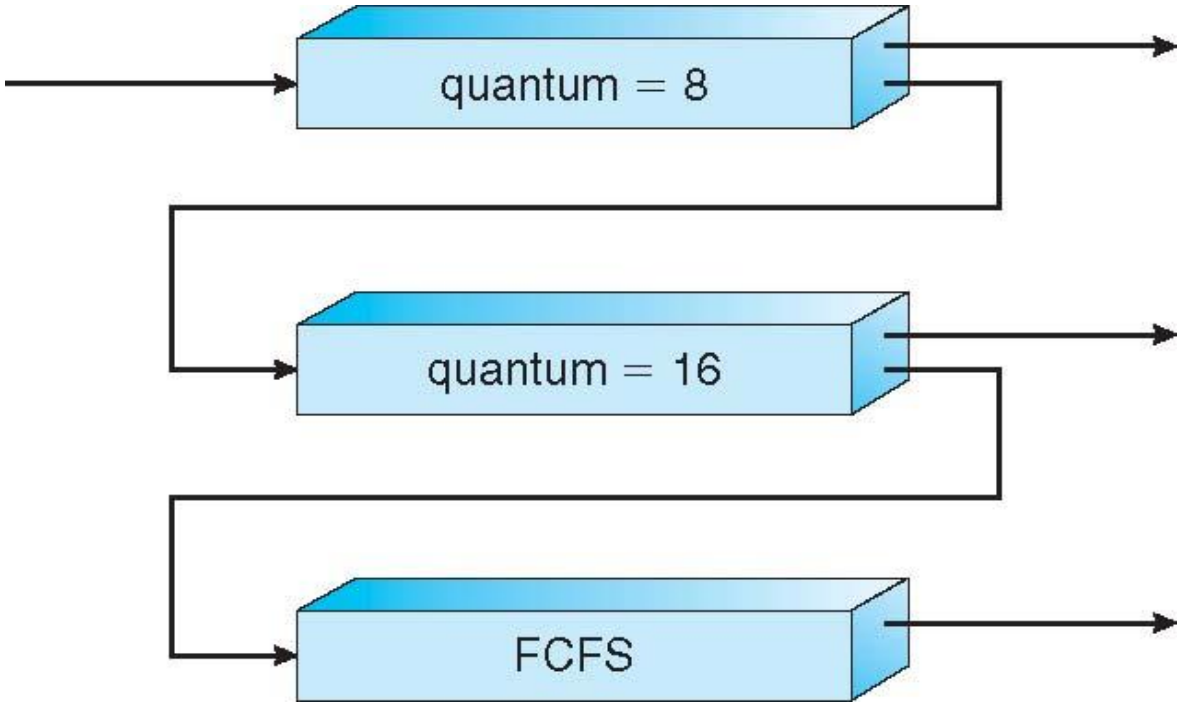
- MFQ scheduling allows the process to move between queues.

- Idea – separate processes according to the characteristics of their CPU bursts.

- Too much CPU time – lower priority queue
- I/O bound and Interactive process – higher priority queue
- Process moves form LPQ to HPQ – aging – starvation.

Example

- Queue 0 – 8 milliseconds
- Queue 1 – 16 milliseconds
- Queue 2 – FCFS when Queue 0 & 1 are empty.



- MFQ Scheduler is defined by the following parameters:

- Number of queues
- Scheduling algorithm for each queue
- Method used to determine when to upgrade a process to a higher priority queue
- Method used to determine when to demote a process to a lower-priority queue
- The method used to determine which queue a process will enter when the process needs service.

Thread Scheduling

- One difference between user level and kernel level threads lies in how they are scheduled.

- The thread library schedules user-level threads to run on an available LWP – PCS – since competition for the CPU takes place among threads belonging to the same process. EX: MtoO, MtoM

- To decide which kernel-level threads to run on a CPU, the kernel uses SCS – competition for the CPU takes place among all threads in the system. Ex: OtoO

Multiple-Processor Scheduling - Approaches

- Asymmetric

- All scheduling decisions, I/O processing, and other system activities handled by a single processor – the master server.
- Other processors execute only user code.
- Simple
- Only one processor accesses the system data structures reduces the need for data sharing.

- Symmetric multiprocessing

- Each processor is self-scheduling.
- Scheduler for each processor examine the ready queue and select a process to execute.
- Scheduler must be programmed carefully.

Issues concerning SMP systems

- Processor Affinity
- Load balancing
- Multicore processors

Processor Affinity

- What happens to cache memory when a process has been running on a specific processor?
- Data most recently accessed – cache

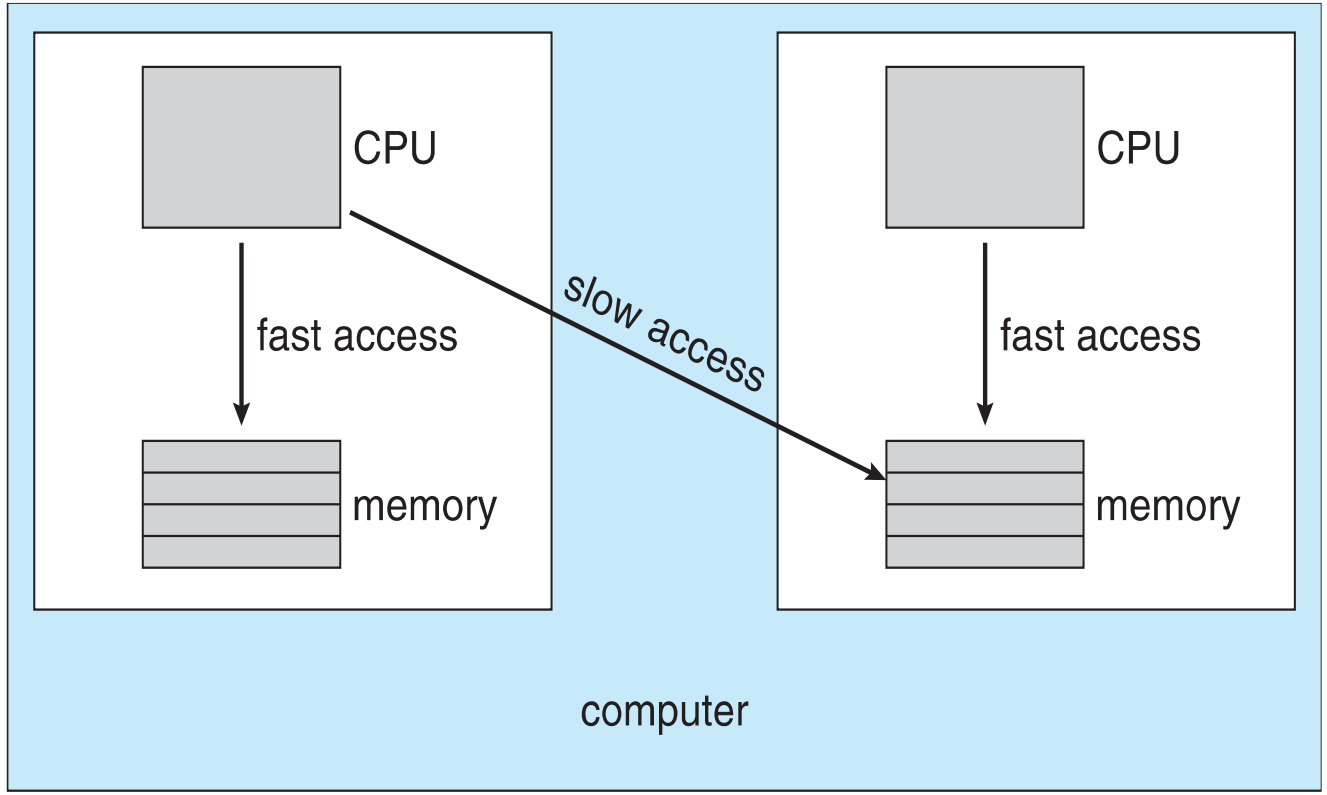
- What happens if the process is migrated to another processor?
- Contents of cache memory must be invalidated for the first processor, and the cache for the second processor must be repopulated.

- Processor Affinity – a process has an affinity for the processor on which it is currently running.

- Soft affinity – Operating system attempt to keep a process on a single processor, but it is possible for a process to migrate between processors.

- Hard affinity – Allows a process to specify a subset of processors on which it may run.

- Main memory architecture of a system can affect processor affinity issues.



Load Balancing

- Workload must be balanced.
- Load balancing attempts to keep the workload evenly distributed across all processors in an SMP systems.
- Load balancing is necessary on systems where each processor has its own private queue of eligible processes to execute.
- It is unnecessary on systems with common run queue.

- Two approaches for load balancing:
- Push migration
- Pull migration

- Load balancing counteracts the benefits of processor affinity.

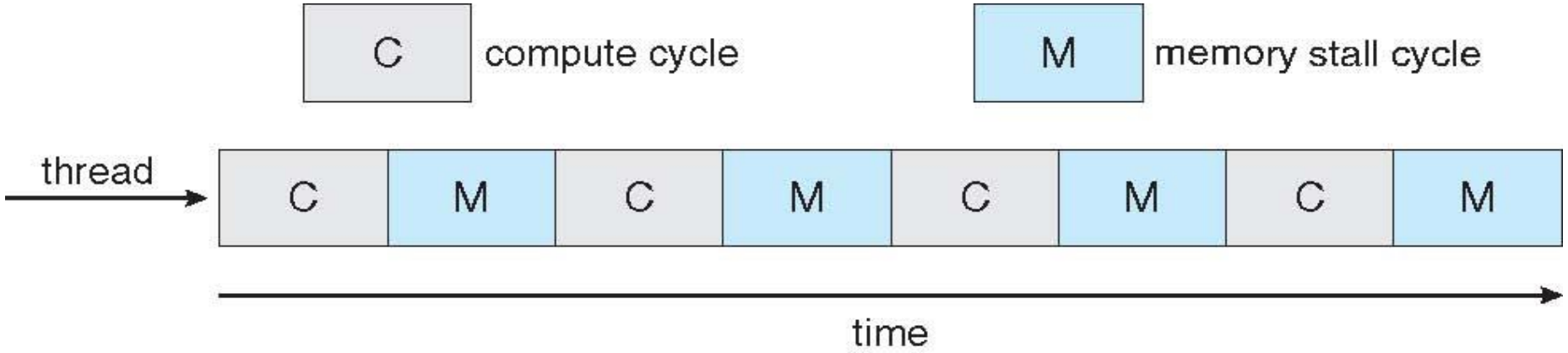
Multicore Processors

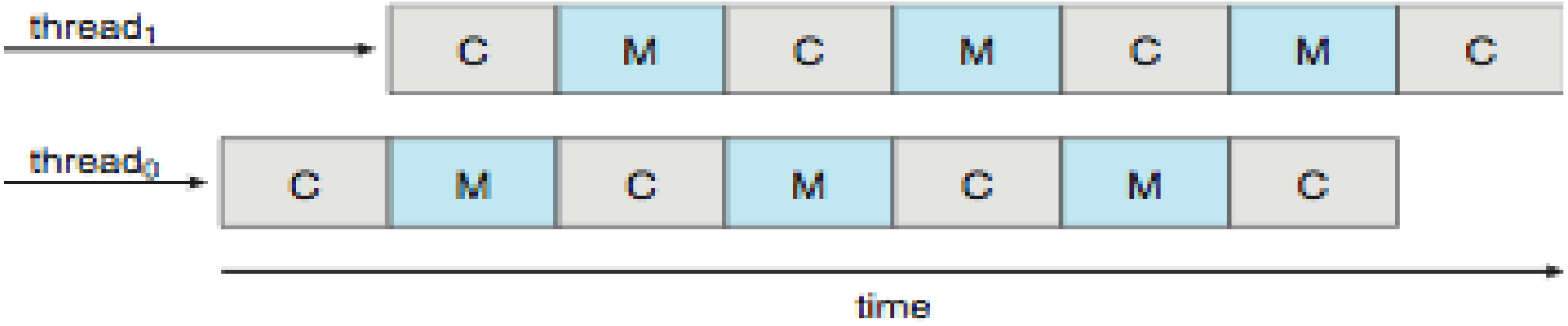
- Multicore processor – place multiple processor cores on the same physical chip.

- SMP systems that use multicore processors are faster and consume less power than systems in which each processor has its own physical chip.

- Multicore processors may complicate scheduling issues:

- Memory Stall – cache miss – solution – multithreaded processor cores.


C
compute cycle
M
memory stall cycle
thread
C
M
C
M
C
M
C
M
time


thread1
C
M
C
M
C
M
C
thread0
C
M
C
M
C
M
C
time

- Two ways to multithread a processing core:

- Coarse-grained – a thread executes on a processor until a long latency event occurs.
- Cost of switching is high

- Fine-grained – switches between threads at a much finer level of granularity.
- At the boundary of each instruction cycle.
- Cost of switching is small as the architecture includes logic for thread switching

- Multithreaded multicore processor requires two level of thread scheduling:

- Which software thread to run on each hardware thread (logical processors)

- How each core decides which hardware thread to run

Real Time CPU Scheduling

- Soft real time systems – provides no guarantee that when a critical process will be scheduled. But guarantees that it will given preference over non critical processes.

- Hard real time systems – task must be serviced by its deadline.

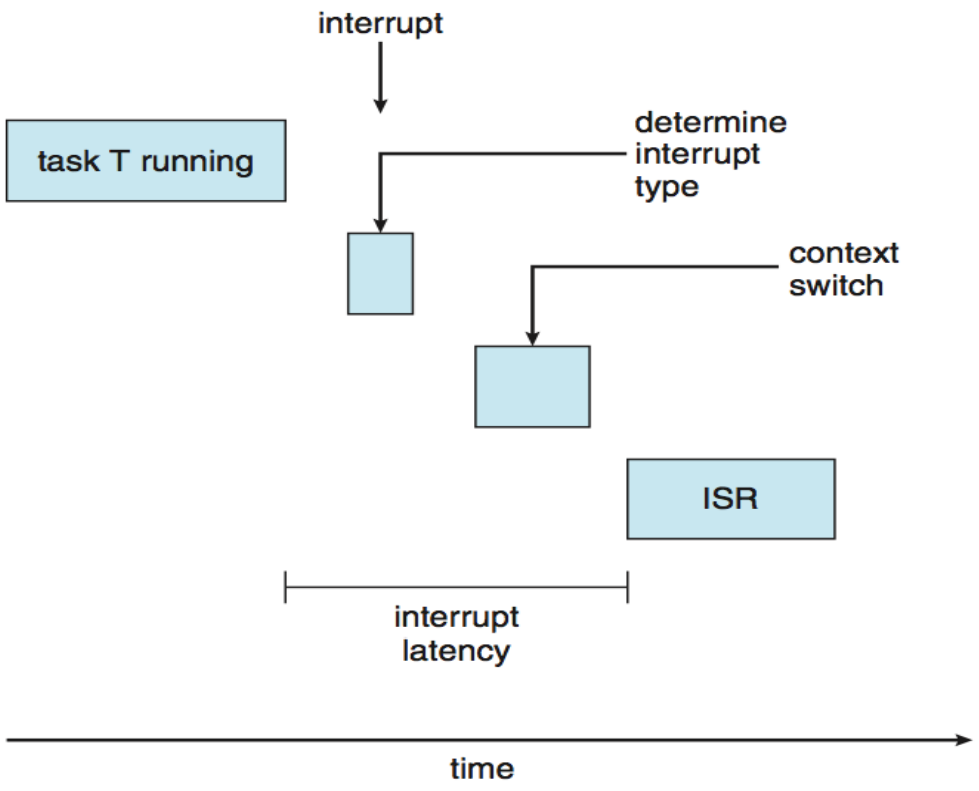
Issues Related to Process Scheduling

- Minimizing Latency

- Event Latency
- Real time systems are even driven in nature – software – hardware
- Time elapses from the time an event occurs to it is servcied.
- Different events have different latency requirements.

- Two types of Latencies affect the performance of the system:

- Interrupt Latency
- Dispatch Latency



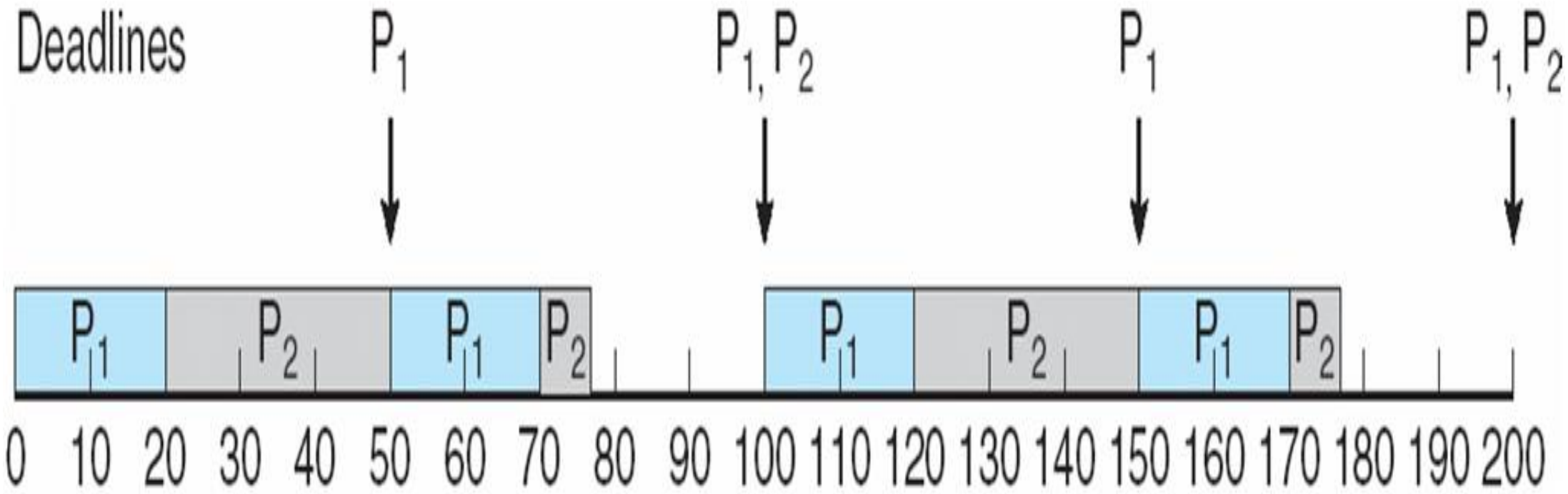
- Priority Based Scheduling
- RTOS should respond immediately to a real-time process – scheduler should support priority based algorithm with preemption.
- Preemptive priority scheduling guarantees soft real time functionality.
- Hard real time systems must guarantee that the real-time tasks will be serviced in accord with their deadline requirements.
- Processes are periodic - require CPU at constant intervals.

Rate Monotonic Scheduling

- Schedules periodic task using static priority policy with preemption.

- Each periodic task is assigned a priority inversely based on its period.

- Two processes P1 and P2.
- P1 → p1= 50, t1 = 20
- P2 → p2=100, t2 = 35

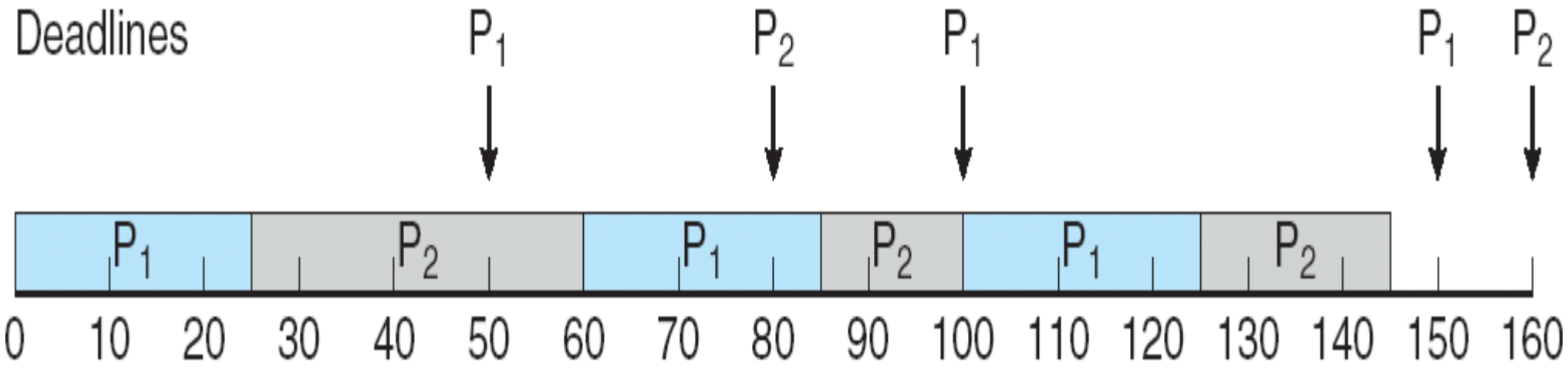

Deadlines
P1
P1, P2
P1
P1, P2
P1
P2
P1
P2
P1
P2
P1
P2
0 10 20 30 40 50 60 70 80 90 100 110 120 130 140 150 160 170 180 190 200

Earliest-Deadline-First Scheduling

- Dynamically assigns priorities according to deadline.

- Earlier the deadline, the higher the priority
- Latter the deadline, the lower the priority

- Two Processes: P1 and P2
- P1 → p1 = 50 and t1 = 25
- P2 → p2 = 80 and t2 = 35


Deadlines
P1
P2
P1
P1
P2
P1
P2
P1
P2
P1
P2
0
10
20
30
40
50
60
70
80
90
100
110
120
130
140
150
160